National University of Lesotho
CS4433 Software Engineering

School Learning Management System

A C++/SQLite Client–Server Application for Offline Assignment Submission, Marking, and Moderation

Submitted by: **Lebohang Lebete**
Student Number: **202321241**
Supervisor: **DR Mamoqenelo Morolong**

3. Project management plan

3.1 Feasibility Study

This section assesses the feasibility of the proposed School Assignment Management System across five dimensions: technical, operational, economic, legal, and schedule.

3.1.1 Technical Feasibility

The system will be implemented as a layered, client–server application over the school's local area network (LAN), satisfying the requirement that the system function without dependence on internet connectivity. A single server process will host the SQLite database and expose functionality to clients over HTTP using JSON payloads, implemented with lightweight, dependency-free libraries (cpp-httplib for HTTP and nlohmann::json for serialisation). This architecture avoids the concurrency and file-locking issues associated with multiple clients accessing a shared SQLite file directly over a network share, since all reads and writes are mediated by a single owning process. Given that the target load is approximately 800 students with predominantly read-heavy access patterns, and that write contention can be mitigated through short-lived transactions in the business logic layer, the system is technically achievable using established, low-risk technologies. Password security will be addressed through salted hashing using a recognised library, and all database access will be parameterised to prevent injection attacks. No unfamiliar or experimental technology is required. The system is assessed as technically feasible.

3.1.2 Operational Feasibility

A key constraint raised by the Principal is that staff are not IT specialists and require a simple system. This is addressed through role-specific, minimal interfaces: each of the four actors (Student, Lecturer, Head of Department, Principal) is presented only with functionality relevant to their role. The underlying workflow; submission, marking, moderation, and release, mirrors the school's existing manual process, meaning the primary adoption barrier is interface complexity rather than process change. Client-side software will be kept deliberately lightweight to accommodate the ageing hardware described by the IT officer. The system is assessed as operationally feasible.

3.1.3 Economic Feasibility

All proposed technologies (C++17, SQLite, cpp-httplib, GoogleTest/Catch2) are open-source and free of licensing cost. As the system operates entirely on-premises, no cloud hosting or subscription costs are incurred. Development effort is the sole cost, and ongoing operational cost is negligible, requiring only a single server machine already available to the school. The system is assessed as economically feasible, with effectively zero marginal cost beyond development time.

3.1.4 Legal Feasibility

The system processes personal data belonging to learners, including assessment marks and submission records, which brings it within the scope of applicable data protection legislation. This is addressed through role-based access control, a complete audit log of all mark changes (recording who made the change, when, the old and new values, and the reason), and the absence of any third-party data sharing, since the system is fully offline and on-premises. These safeguards will be formalised as non-functional requirements in the Software Requirements Specification. The system is

assessed as legally feasible, provided data protection and auditability are treated as first-class requirements rather than implementation afterthoughts.

3.1.5 Schedule Feasibility

The project must be delivered by a single developer across seven weeks, following the phase allocation specified in the project brief. This timeline is achievable provided architectural decisions are finalised early and not revisited during implementation. The principal schedule risk lies in Phase 3 (System Design), which carries the largest mark allocation (24 marks) but is allotted only one to two weeks, while requiring the greatest number of artefacts (architecture documentation, detailed class diagram, three behavioural diagrams, and database design). This risk is carried forward into the project risk register. The system is assessed as feasible on schedule, though time-constrained.

3.1.6 Recommendation

On balance, no feasibility dimension precludes the project. The recommendation is to proceed, with active management of three identified watch-items: (1) maintaining interface simplicity for non-technical users, (2) ensuring data protection compliance for student records, and (3) mitigating time pressure in Phase 3 through early architectural commitment.

3.2 Work Breakdown Structure and Schedule

3.2.1 Work Breakdown Structure

The project is decomposed into eight phases, consistent with the project brief and the Waterfall Model adopted for this project. Each phase is broken down into concrete work packages below.

1. Project Planning and Management
1.1 Feasibility study
1.2 Work breakdown structure and Gantt chart
1.3 Risk register

2. Requirements Engineering
2.1 Stakeholder analysis and conflict resolution
2.2 Software Requirements Specification (functional and non-functional requirements)
2.3 Use case diagram
2.4 Fully dressed use case specifications (three use cases, including "Submit assignment")

3. System Design
3.1 Architectural style selection and justification, component diagram
3.2 Detailed class diagram (with design patterns applied)
3.3 State machine diagram (Submission life cycle)
3.4 Activity diagram with swimlanes (submission to released mark)
3.5 Sequence diagrams ("Submit assignment", "Moderate and release marks")
3.6 Database design: ERD, data dictionary, SQL schema

4. Implementation
4.1 Data access layer (Repository/DAO, SQLite integration)
4.2 Abstract User hierarchy and role-based logic

4.3 Business logic (late-penalty, extension, resubmission, moderation rules)
4.4 HTTP API layer (server)
4.5 Client application(s)
4.6 Authentication and password security
4.7 Audit logging
4.8 Build system (CMake) and coding standard documentation

5. Verification and Validation
5.1 Test strategy and test plan
5.2 Automated unit and integration tests
5.3 System and acceptance testing
5.4 Defect log

6. Configuration Management and Quality Assurance
6.1 Git repository setup, branching strategy, CI pipeline
6.2 Static analysis and code metrics
6.3 Self code review

7. Deployment
7.1 Deployment diagram
7.2 Installation guide
7.3 User manuals (per role)
7.4 Release notes (v1.0)

8. Maintenance
8.1 Change request (CR1) classification and impact analysis
8.2 Implementation of CR1 (v1.1)
8.3 Regression testing

3.2.2 Schedule and Gantt Chart

The schedule follows the timeline specified in the project brief, mapped against the work packages above. Weeks are numbered from the start of the project.

Work Package	Wk1	Wk2	Wk3	Wk4	Wk5	Wk6	Wk7
1.Planning and management	■						
2. Requirements Engineering		■	■				
3. System design			■	■			
4. Implementation				■	■		
5. Verification and validation					■	■	
6. Configuration management and QA				■	■	■	
7. Deployment						■	
8. Maintenance							■
Completion (report, viva)							■

Milestones

- Gate 1 (end of Week 2): Requirements engineering complete, SRS and use case model reviewed.
- Gate 2 (end of Week 3): System design complete and approved, implementation may not begin before this point.
- Gate 3 (end of Week 6): Verification and validation complete.

3.2.3 Dependencies and Critical Path

The following dependencies constrain the schedule:

- Requirements engineering (2) must precede system design (3), since the class diagram, database design, and use cases all derive from the SRS.
- System design (3) must pass Gate 2 before implementation (4) may begin, per the project brief's explicit constraint.
- Implementation (4) must precede verification and validation (5), since automated tests are written against the implemented business logic layer.
- Configuration management (6) runs in parallel with implementation and V&V, since the CI pipeline must be in place before automated tests can be continuously run, but static analysis and code review depend on implementation being substantially complete.
- Deployment (7) depends on V&V (5) passing Gate 3.
- Maintenance (8) depends on deployment (7), since CR1 is implemented against the released v1.0 system.

The **critical path** runs through: Requirements Engineering → System Design → Implementation → Verification and Validation → Deployment → Maintenance → Completion. This path leaves no slack, since each phase gate is a hard prerequisite for the next; the project's tightest constraint is the one-to-two-week allocation for System Design (Section 3.2.1, work package 3), which produces the largest number of artefacts relative to time available and is therefore flagged as the primary schedule risk in Section 3.3.

3.3 Risk Register

The risk register below identifies risks across technical, project, and people categories, consistent with the requirement to cover at least six risks spanning all three. Exposure is calculated as Likelihood × Impact, each rated on a scale of 1 (low) to 5 (high).

ID	Category	Risk Descripion	Likelihood	Impact	Exposure	Mitigation	Owner
R1	Technical	SQLite write contention or corruption under concurrent	2	4	8	Route all writes through a single owning server process; keep transactions short;	Lebohang Lebete

		access during peak periods (e.g. deadline day)				wrap all database operations in exception handling with rollback	
R 2	Technica l	Password security implementation flaw (e.g. weak hashing, plain-text fallback)	2	5	10	Use a recognised, well-audited hashing library (e.g. bcrypt/libsodium) rather than a custom implementation; include a dedicated security test case	Lebohan g Lebete
R 3	Project	Phase 3 (System Design) is allocated only 1–2 weeks but carries the largest artefact set and mark weighting (24 marks)	4	5	16	Begin architecture and class diagram drafts early, in parallel with finishing requirements; timebox each design artefact; use design decisions already agreed (Section 3.2) to avoid rework	Lebohan g Lebete
R 4	Project	Scope creep: attempting to gold-plate the client UI or add features beyond the SRS's Must requirements	3	3	9	Strictly prioritise using MoSCoW in the SRS; defer all Should/Could items until Must requirements are complete and tested	Lebohan g Lebete
R 5	Project	Underestimatin g time required for automated test coverage (70% target on business logic	3	3	9	Write unit tests incrementally alongside business logic implementation rather than after the fact; track	Lebohan g Lebete

						coverage weekly via CI	
R 6	People	Solo developer: no backup if illness, technical blocker, or personal emergency causes lost time	3	5	15	Build schedule slack into non-critical-path tasks; commit and push work daily so partial progress is never lost; keep a personal risk log of blockers as they arise	Lebohang Lebete
R 7	People	Loss of motivation or burnout given the volume of deliverables across 7 weeks alongside other coursework	2	3	6	Break work into the small, concrete tasks shown in the WBS (Section 3.2.1) rather than large undifferentiated phases; review progress against the Gantt chart weekly	Lebohang Lebete
R 8	Technical	Misalignment between the database schema and the UML class diagram, reducing traceability marks	2	3	6	Design the class diagram and ERD together in Phase 3, explicitly documenting and justifying any differences as required by the brief	Lebohang Lebete

Risk Register Review Policy

This register will be reviewed and updated at each of the three phase gates (end of Weeks 2, 3, and 6), with new risks added as they emerge and closed risks marked as resolved. The highest-exposure risks at project outset are **R3** (System Design time pressure) and **R6** (solo-developer continuity), both of which are addressed structurally through early design work and daily incremental commits rather than end-of-phase catch-up.